

배열 완성 작업에서 코드 생성 모델의 선호와 예측 단서 분석: 리액트 hooks 함수-배열 구조를 중심으로

박준영

인문대학 언어학과

서울대학교

bloomwayz@snu.ac.kr

초록

하수는 두 산 틈에서 나와 돌과 부딪쳐 싸우며, 그 놀란 파도와 성난 물머리와 우는 여울과 노한 물결과 슬픈 곡조와 원망하는 소리가 굽이쳐 돌면서, 우는 듯, 소리치는 듯, 바쁘게 호령하는 듯, 항상 장성을 깨뜨릴 형세가 있어, 전차 만승과 전기 만대나 전포 만가와 전고 만좌로써는 그 무너뜨리고 내뿜는 소리를 족히 형용할 수 없을 것이다. 모래 위에 큰 돌은 홀연히 떨어져 섰고, 강 언덕에 버드나무는 어둡고 컴컴하여 물지킴과 하수 귀신이 다투어 나와서 사람을 놀리는 듯한데, 좌우의 교리가 붙들려고 애쓰는 듯싶었다. 혹은 말하기를, “여기는 옛 전쟁터이므로 강물이 저같이 우는 것이다” 하지만 이는 그런 것이 아니니, 강물 소리는 듣기 여하에 달렸을 것이다.

1 서론

대형 언어 모델 *large language models (LLMs)*의 프로그래밍 성능은 계속해서 향상되어 왔으며, 최근 모델은 코드 생성 작업에서 강한 성능을 보인다(Austin et al., 2021; Fried et al., 2023). 이러한 성능에 힘입어 인공지능 도구는 이미 개발 현장에 폭넓게 자리 잡아 대부분의 개발자가 인공지능 도구를 개발에 활용하고 있고, 그중 많은 이들이 인공지능 도구의 성능에 호의적이라고 밝혔다(Stack Exchange, Inc., 2025).

이 추세를 따라 코드 생성 모델의 행동과 내부 구조에 대한 연구도 다방면으로 이루어졌다. 대표적으로 모델이 코드의 문법 구조, 식별자, 데이터 흐름, 제어 흐름, 기능적 동등성과 같은 정보를 어느 정도 표상하는지 분석한 연구가 있고(Troshin and Chirkova, 2022; Ma et al., 2024; Maveli et al., 2025), 코드 모델의 예측이 변수명이나 의미 보존 변형과 같은 표면적 변화에 민감할 수 있음을 보인 연구도 있다(Yefet et al., 2020; Wang et al., 2024; Orvalho and Kwiatkowska, 2025). 이러한 연구들은 코드 모델의

예측이 프로그램 의미뿐 아니라 다양한 표면적·구조적 단서의 영향을 받을 수 있음을 시사한다.

다만 이들 연구는 주로 완성된 코드의 정답 여부나 오류 유형, 의미 보존 변형에 대한 강건성, 모델 표상의 전반적인 성질을 분석하는 데 초점을 두었다. 특정 위치에서 후보 토큰의 로짓이 어떤 단서에 따라 달라지는지는 아직 충분히 분석되지 않았으며, 특히 라이브러리의 의미 규약과 일반적인 문법 패턴이 함께 작동하는 상황에서 모델이 다음 토큰을 예측할 때 어떤 단서에 기대는지를 국소적으로 살펴본 연구는 찾아보기 어렵다.

이에, 이 글은 대형 언어 모델이 코드의 다음 토큰을 예측할 때 어떤 토큰을 선호하는지 관찰하고, 이때 어떤 정보가 어느 정도로 코드 생성에 영향을 미치는지 분석한다. 특히 이 연구에서는 리액트 *React* 라이브러리에서 제공되는 함수 가운데 하나인 *useEffect*의 구조와 유사한 함수-배열 패턴에 모델이 어떻게 반응하는지를 집중적으로 살펴본다. 아래는 우리가 연구에서 사용한 함수-배열 패턴의 예시이다.

```
1 const [s, setS] = useState(0);
2 const c = 0;
3
4 useEffect(() => {
5   fetch(`/api/me?x=${s}&y=${c}`)
6 }, [s])
```

우선 위 코드에 등장하는 변수를 살펴보자. 첫 번째 줄에서는 상태 변수 *s*와 상태를 갱신하는 함수 *setS*를 선언하고 있고, 두 번째 줄에서는 상수 *c*를 선언하고 있다. 상태 *s*와 상수 *c*는 얼핏 닮아 보이지만, 상태는 갱신 함수에 의해 그 값이 달라질 수 있고, 상수의 값은 프로그램이 종료될 때까지 값이 달라지지 않는다는 점에서 둘은 대별된다.

그 다음 집중할 것은 *useEffect* 함수를 호출하는 대목이다. *useEffect*는 또 다른 함수 *f*와 배열 *[*i*]*

를 인자로 받는 함수인데, 이때 배열은 상태 등의 반응형 *reactive* 값만으로 구성될 것이 강력히 권고된다 (Meta Platforms, Inc., 2026). 상태 *s*와 상수 *c*는 모두 함수의 본문에 등장하지만 배열에는 *s*만이 존재하는데, 이는 *s*는 상태 변수로서 반응형 값이고 *c*는 상수로서 비반응형 값이기 때문이다.

여기서 몇 가지 의문이 남는다. 대형 언어 모델은 함수-배열 구조에서 배열에 어떤 항목이 올지 올바르게 예측할 수 있을까? 올바르게 예측할 수 있다면, 혹은 예측이 틀리다면, 모델은 어떤 정보를 근거로 삼아 토큰을 예측했을까? 모델이 리액트 라이브러리를 사용하여 코드가 작성되었다는 점을 인식하고 리액트 훅의 의미를 계산했기 때문일까, 아니면 함수와 배열이 매개변수로 쓰이는 문법 구조 때문일까? 그것도 아니라면, *useState* 같은 키워드를 단서로 삼는 것일까?

이 질문에 답하기 위해, 우리는 45개의 변수 쌍을 바탕으로 자바스크립트 코드 데이터셋을 구축하였다. 각 변수 쌍에 대해 동일한 코드 골격을 유지한 채 함수 이름, 변수 선언 형태, 변수 선언 순서, 변수 이름을 조작하여 40개의 변형 코드를 만들었다. 이 데이터를 이용하여 두 언어 모델 Llama3.2-1B와 Gemma3-1B가 배열의 첫 항목을 예측하도록 했고, 이때 예측되는 토큰의 로짓을 측정하였다. 이어서 함수 본문에서의 변수 사용과 배열을 도입하는 문맥을 각각 조작하는 두 건의 소거 *ablation* 실험을 통해 관찰된 선호가 무엇으로 환원되는지를 분해하였다.

실험 결과, 우리는 다음 세 가지를 발견할 수 있었다.

- **상태 변수 선호.** 모델은 실험 조건 전반에서 배열의 첫 요소로서 상태 변수를 선호하는 모습을 보였다. 이 경향은 리액트 훅 *useEffect* 또는 *useLayoutEffect*가 사용되었는지, 일반 자바스크립트 함수 *subscribe*가 사용되었는지에 상관없이 일관되게 나타나며, 오히려 리액트 환경일 때보다 일반 자바스크립트 환경일 때 모델이 상태 변수를 선호하는 정도가 더 높았다.
- **선언 형태와 변수 사용 단서.** 이러한 모델의 선호는 두 가지 단서가 더해진 결과로 보인다. 하나는 변수가 *useState* 형태로 선언되었다는 ‘선언 형태’ 단서이고, 다른 하나는 그 변수가 함수 본문에서 실제로 쓰였다는 ‘본문 사용’ 단서이다.

두 단서는 대체로 합쳐지는 방향으로 작동하지만, 본문에서 상태 변수가 쓰이지 않은 상황에서 어느 단서가 더 강하게 작용하는지는 배열을 도입하는 문맥이 리액트 환경인지의 여부에 따라 달라진다.

- **배열로의 일반화.** 이 선호는 리액트의 의존성 배열에서만 나타나는 것이 아니다. *useEffect*나 *subscribe*처럼 함수와 배열을 인자로 전달하는 상황이 아니라 독립적인 배열을 완성할 때에도 똑같이 상태 변수를 더 선호한다. 따라서 모델이 상태 변수를 선호하는 현상은 함수-배열 패턴에서만 일어나는 것이 아니라 모델이 배열을 채우는 일반적 경향에 가깝다.

참고 문헌

- Jacob Austin, Augustus Odena, Maxwell I. Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, and Charles Sutton. 2021. [Program synthesis with large language models](#). *CoRR*, abs/2108.07732.
- Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Scott Yih, Luke Zettlemoyer, and Mike Lewis. 2023. [In-coder: A generative model for code infilling and synthesis](#). In *The Eleventh International Conference on Learning Representations*.
- Wei Ma, Shangqing Liu, Mengjie Zhao, Xiaofei Xie, Wenhan Wang, Qiang Hu, Jie Zhang, and Yang Liu. 2024. [Unveiling code pre-trained models: Investigating syntax and semantics capacities](#). *ACM Transactions on Software Engineering and Methodology*.
- Nickil Maveli, Antonio Vergari, and Shay B. Cohen. 2025. [What can large language models capture about code functional equivalence?](#) In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 6880–6918. Association for Computational Linguistics.
- Meta Platforms, Inc. 2026. Separating events from effects – react. <https://react.dev/learn/separating-events-from-effects>. Accessed: 2025-06-23.
- Pedro Orvalho and Marta Kwiatkowska. 2025. [Are large language models robust in understanding code against semantics-preserving mutations?](#) *Preprint*, arXiv:2505.10443.
- Stack Exchange, Inc. 2025. 2025 stack overflow developer survey. <https://survey.stackoverflow.co/2025/>. Accessed: 2026-06-23.

- Sergey Troshin and Nadezhda Chirkova. 2022. [Probing pretrained models of source codes](#). In *Proceedings of the Fifth BlackboxNLP Workshop on Analyzing and Interpreting Neural Networks for NLP*, pages 371–383. Association for Computational Linguistics.
- Zhilong Wang, Lan Zhang, Chen Cao, Nanqing Luo, Xinzhi Luo, and Peng Liu. 2024. [How does naming affect llms on code analysis tasks?](#) *Preprint*, arXiv:2307.12488.
- Noam Yefet, Uri Alon, and Eran Yahav. 2020. [Adversarial examples for models of code](#). *Proceedings of the ACM on Programming Languages*, 4(OOPSLA):1–30.